1. Pitch

Offline-first medical scribe that is designed for healthcare workers in remote areas with little to no wifi. It captures doctor and patient conversations and structures them into clinical notes all on device. It has context of patients' previous visits to help support doctor's decisions about certain medications.

2. Core Architecture

Options	Pros	Cons
Fully On Device	• **Absolute Privacy:** Patient data never leaves the handset, ensuring HIPAA compliance by design. • **Zero Latency & Dependency:** Works seamlessly in remote clinics with zero cellular or Wi-Fi connectivity. • **No Infrastructure Costs:** Eliminates expensive recurring cloud hosting and LLM API fees.	• **Hardware Constraints:** Strictly bound by local handset limitations (RAM, NPU thermal throttling, and battery drain). • **Data Redundancy Risks:** High risk of permanent data loss if the physical device is broken, lost, or stolen.
On device + cloud sync	• **Data Redundancy:** Secure backups protect critical clinical notes from local hardware failures. • **Multi-Device Ecosystem:** Enables seamless synchronization, allowing notes to be viewed on desktop EHR systems later.	• **Security Complexity:** Requires building a complex end-to-end encryption (E2EE) pipeline to keep data safe in transit and at rest. • **Connectivity Dependent:** Backup features break entirely down in low-connectivity areas.

		• **Hosting Overhead:** Requires setting up and maintaining cloud infrastructure.

- Focus on Fully on Device First
- If time permits, we can implement a cloud sync feature

3. AI Pipeline

Model	**What it Does**	**Options**		
Audio Model	Listens to ambient, messy acoustic room audio and streams it into raw text waveforms.	**whisper-small.en** https://github.com/openai/whisper https://huggingface.co/openai/whisper-small.en	Google MedASR https://developers.google.com/health-ai-developer-foundations/medasr	
Embeddings	Embed word to vectors	**MiniLM** https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2		
LLM	Reads the raw transcript, extracts historical database contexts, filters noise, and outputs a structured JSON SOAP note	Google MedGemma 1.5 Medically tuned but might be harder to implement with executorch	**Llama 3.2 1B (Quantized to INT8 via torch.ao)** Not medically tuned, but better integration with Executorch	

4. Patient Data and Local Memory

Memory Type	Pros	Cons	
Vector Embeddings	Understands medical synonyms and context.	Difficult to compile. Requires running a second AI model just to generate the math vectors.	
Regular Text	Zero complexity. No vector models required. Easy to query.	raw text into the LLM prompt massively slows down inference	
SQLite keyword matching	takes up less space	Will miss context if the exact spelling or phrasing isn't used.	
Sqlite-vec (SQL and vector db search combined.)	Sql and vector db search Keeps all data elegantly in one single `.db` file. Pure C extension (easy Android NDK compile).	Still need to use an embedding model	https://github.com/asg017/sqlite-vec

5. Mock Datasets

- Synthea
- **Faker** (python library) - Our pick since its not a big dataset, easier to demo

6. Executorch Deployment

- Off Device Compilation
    - **Post-Training Quantization (PTQ):** We utilize Meta's `torch.ao` library to compress the full-precision weights of Whisper, MiniLM, and Llama 3.2 down to 8-bit integers (`INT8`).

    - **Graph Export:** Python dependencies are entirely stripped from the models, tracing them into static mathematical graphs.
    - **Output:** Three highly compressed, serialized binary files (`.pte` format) are generated and bundled into the Android application assets.
- On-Device Execution (Live Patient Visit)
    - **ExecuTorch Runtime):** Our native C++ logic invokes the ultra-lightweight ExecuTorch API to load the `.pte` binaries into the device memory strictly on-demand.
    - ExecuTorch statically allocates the exact RAM required for inference before execution begins, ensuring the Android Low Memory Killer (LMK) is not triggered.
    - **The Hardware Bridge (Qualcomm QNN Delegate):** Using the QAIRT SDK, ExecuTorch bypasses the device CPU and delegates the mathematical tensor operations directly to the **Snapdragon Hexagon NPU** for maximum speed and battery efficiency.

7. Roles

AI Pipeline Developers

- Team Member 1 - **Quantization -** have to ensure that the quantization settings for the models (Whisper/MiniLM/Llama) actually fit into the static memory allocation allowed by the ExecuTorch runtime. tweaking the compression settings so that it fits for the device
- Team Member 2 - **Hardware** - In charge of the `QAIRT SDK` and `Android NDK`. challenge is getting the model to stay on the NPU without falling back to the CPU. If the model starts running on the CPU, the app will lag; this person's job is to ensure 100% of the mathematical tensor operations remain on the Snapdragon Hexagon hardware.

Local Storage/Memory Developers

- Team Member 3 - **Vector & Memory -** handling the C-based integration of the sqlite-vec extension into the Android build system. dealing with binary data, pointer management, and ensuring that the vector similarity search is actually performing calculations correctly in C without memory leaks.
- Team Member 4 - **Reading & Prompting -** designing the system prompt that forces Llama 3.2 to correctly integrate the historical context retrieved by Coder 3. They have to solve the inference bottleneck: if they feed too much historical text into the LLM, the model will run too slowly. They have to code the logic to prune and select only the most relevant clinical history to keep the Time-to-First-Token fast.

https://claude.ai/share/e8a4b145-04c5-43a3-8985-263cc69be3f8

8. UI/UX Flow

Developer 1: The Hardware & Runtime Lead

- **Focus Areas in the Prompt:** Section 3-A (`simple-whisper-transcription`), Section 3-C (`simple_npu_chatbot`), and Section 3-D (Qualcomm SDKs).
- **Today's Mission:** You own the Android NDK and hardware execution. Your job is to take the compiled `.pte` model binaries and use the Qualcomm QNN delegate to make sure they run natively on the Snapdragon NPU.
- **Your Blueprint:** Use the `simple-whisper-transcription` and `simple_npu_chatbot` repositories as your starter templates. They already have the boilerplate code written to hook up the microphone and talk to the NPU. Copy their setup.

Developer 2: The Python Pipeline & Logic Architect

- **Focus Areas in the Prompt:** Section 3-A and 3-B (Hugging Face Model Weights) and Section 3-C (`local-agent`).
- **Today's Mission:** You own the data prep and the "brain" of the app. You are responsible for running the Python compilation scripts to compress the models down to `INT8` and export them as `.pte` files. Once you hand those files to Developer 1, pivot immediately to writing the system prompt and the RAG orchestration logic.
- **Your Blueprint:** Use the `local-agent` repository. Look at how it structures an offline agent loop to intercept user input, pause, look up data, and feed it back to the LLM.

Developer 3: The Local Storage Architect

- **Focus Areas in the Prompt:** Section 3-B (`sqlite-vec`) and Section 3-E (`Pose-Detection-with-HRPoseNet`).
- **Today's Mission:** You own the local memory. Your job is to compile the `sqlite-vec` C-extension for Android, build the local SQL database schema, and create the virtual tables that will hold the patient history vectors.
- **Your Blueprint:** Use `sqlite-vec` to handle the distance math. Because you are compiling a raw C-extension into Android, use the `Pose-Detection-with-HRPoseNet` repo purely to see how Qualcomm engineers configure their Android CMakeLists.txt and folder trees for native code. Once your database is stable, jump in and help Developer 1 hook up the JNI layer so the UI can read your tables.

9.